X BY 2®

X BY 2 · AI-ASSISTED DELIVERY

Specs, Not Prompts

How an ordinary development team built a real platform with AI agents



X by 2 · November 2026

Could five people build a real platform with AI agents writing most of the code?

And could they do it in a way another team could understand, repeat and audit?

GOAL 1 · THE PRODUCT

Ship an internal test-automation platform

A hosted, self-service way to run approved Playwright scripts from a form, with every run kept as evidence. Built for QA engineers, business users, developers and pipelines.

GOAL 2 · THE PROCESS

Prove a repeatable way to build with AI

A documented loop that people and agents both follow, that leaves an audit trail, and that other teams can adopt.

Started 9 Aug 2026

Median build time: 7 days → 1.9 days

An ordinary team and two off-the-shelf AI agents

AN ORDINARY TEAM



Five developers	no AI specialists
Off-the-shelf agents	no special tooling
Four on Claude Code	Claude Code
One on Codex	Codex

Same process, two different agents.

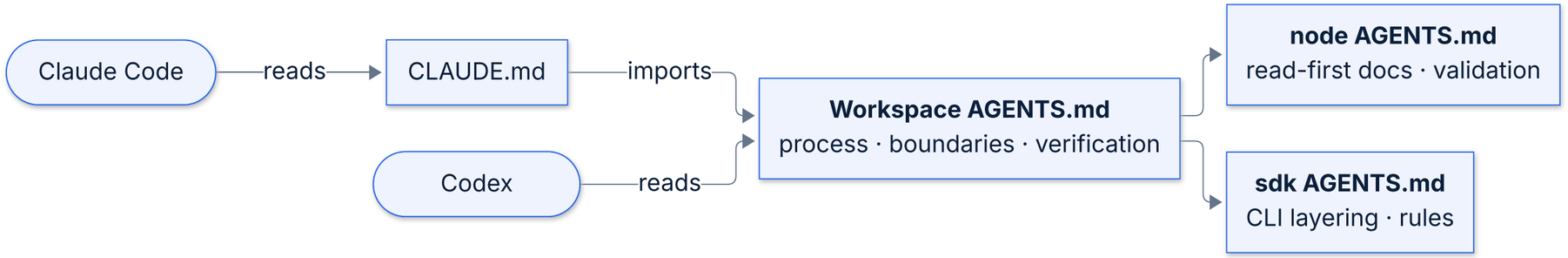
WHY THIS MATTERS

If an ordinary team can do it, another team can repeat it

No AI specialists, no special tooling, and two different agents following the same process. The process shouldn't depend on one tool or one person.

Four repositories: three hold product code, one holds the process. That fourth one is next.

Agents get an onboarding guide, just like new hires



- **One source of truth:** `CLAUDE.md` just imports `AGENTS.md`, so both agents get the same rules
- **Short:** three guides, under 200 lines in total, pointing to the real docs and ADRs
- **Versioned:** changed through stories and pull requests, like code

AGENTS.md — workspace · excerpts

```
Every implementation change … flows through the SDLC
loop defined in jira/README.md

Nothing is implemented from an unapproved spec.

Small changes still get a story - a small change just
gets a small spec.

Run the verification command of every repository changed
by a task before presenting work
```

A workspace repository sits above the code repositories

THE REPOSITORIES

platform-app
The platform: Dashboard, API, shared contracts and persistence

platform-sdk
The authoring SDK and local runner CLI

platform-runner
The Jenkins pipeline that executes runs, split into its own repository on 15 Sep

process-workspace
The process itself: stories, specs, skills and agent guides

START AT THE TOP

Every AI session starts at the workspace level, so while refining, the agent can see all three code repositories.

BUILD ONE REPO AT A TIME

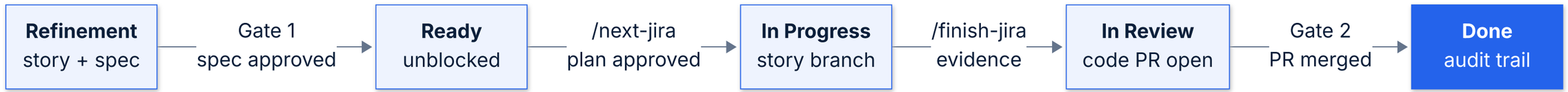
Refinement spans repositories, but each story is scoped to a single one, so the agent only changes one codebase per task.

LINK THE REST

When a change spans repositories, it becomes separate stories with dependencies between them.

Repeatable: the process lives in a repository any team can adopt.

Every change follows one loop, with two human gates



Nothing is built from an unapproved spec

The team resolves the spec's open questions before anyone writes code.

Small changes still get a story

A small change gets a small spec, not no spec.

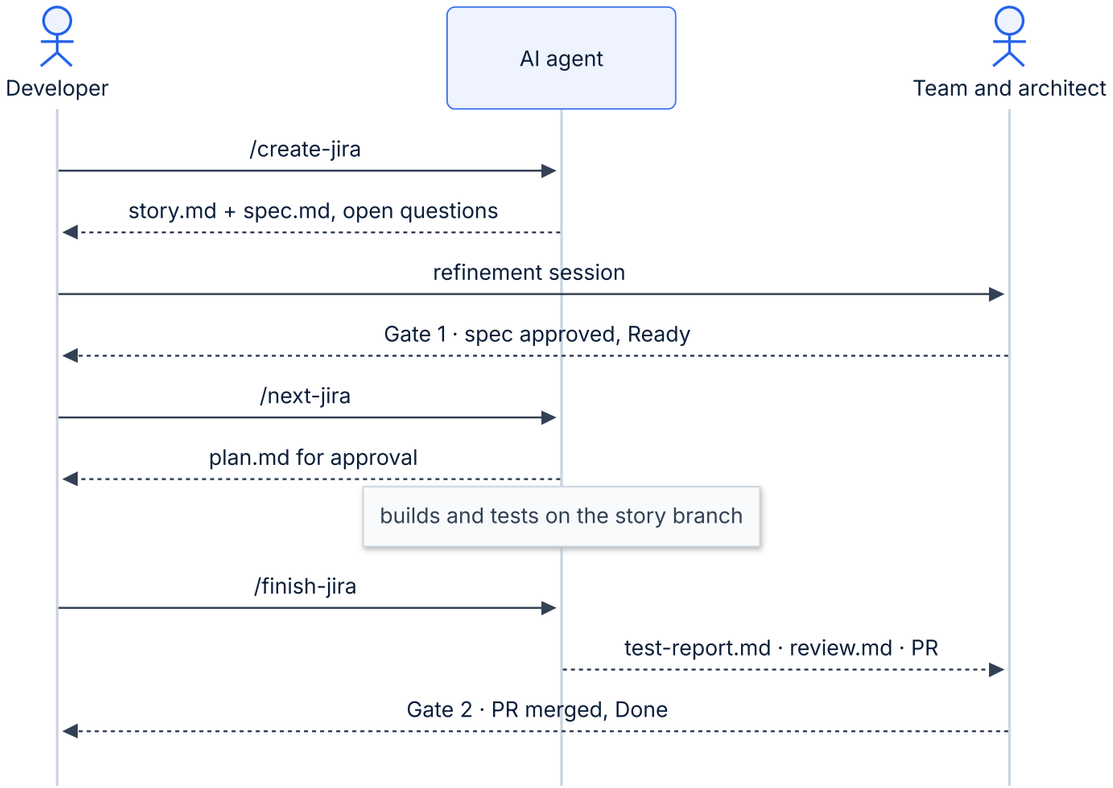
The story folder is the audit trail

Story, spec, plan, test report and review stay with the work. Won't Do is the off-ramp.

WHY A TICKET FOLDER Our environment had no Jira for the agents, so `jira/` in the workspace plays that role. Statuses, dependencies and hand-off are what matter: if a session is lost, the agent picks up where it stopped.

People decide; agents draft, build and check

You own your spec. AI makes refinement produce more text, not less work: produce all the slop you like, just don't throw it at people.



PEOPLE OWN

- What to build
- The decisions in the spec
- Approving the plan
- Approving the merge

AGENTS OWN

- First drafts of story and spec
- The plan and the code
- Tests and verification
- Evidence: test report, review, PR

The process is packaged as skills the agents run



LIFECYCLE SKILLS

Three commands, one per stage a person drives: draft a story, build it, finish it.

STANDARDS SKILLS

Reusable "how to do it well" instructions, so every story, spec and PR looks consistent.

MOST IMPORTANT TWO

Using a ticket system, and writing a spec. How you write the spec matters less than having one way, so every spec looks the same.

Skills carry team knowledge, travel to new projects and work for both agents

DOMAIN SKILL

design-system-ui

Teaches agents the organization's design system. This is what made the redesign in the case study possible: the agent built mockups on the real stylesheet.

PORTABLE

sdlc-bootstrap

Installs the standards in a new project and interviews the team to generate that project's own lifecycle skills. The generic parts travel; the specifics are generated.

BOTH AGENTS

One source, two tools

`.claude/skills` is the single source. A sync script mirrors it for Codex, so changing a skill changes it for everyone.

Ten skills in all: three lifecycle, five standards, one domain, one setup. The appendix lists them.

Three commands take a story from idea to merge

process-workspace — agent session

```
› /create-jira Redesign the Dashboard layouts from the approved mockups
  ✓ jira/story-a-ui-redesign/story.md and spec.md drafted on story-a-refinement
  ✓ open questions listed for the refinement session

› /next-jira platform-app
  ✓ Story A is Ready and unblocked. Pick it up? yes
  ✓ plan.md written: 14 tasks. Approve? yes
  … implementing on branch story-a-ui-redesign

› /finish-jira
  ✓ npm run verify · npm run test:ui (140 unit, 29 UI tests passing)
  ✓ test-report.md and review.md recorded (adversarial review: 2 issues fixed)
  ✓ PR opened: Story A - implementation
```

ILLUSTRATIVE Reconstructed from the skill definitions and Story A's artifacts, not a recorded session.

A full UI redesign: one developer, four working days

THU 24 SEP

- 12:45 — AI builds 8 clickable mockup pages — AI
- 13:50 — Second iteration after design review — AI

Fri 25 – Mon 28 · mockups reviewed, story and spec drafted

TUE 29 SEP

- 09:00 — Open questions decided · spec approved, Ready — PEOPLE
- 15:05 — plan.md: 14 tasks, spec gaps listed for approval — AI

WED 30 SEP

- 08:09 — Implementation: 43 files changed — AI
- 09:51 — Test report and adversarial review — AI
- 09:58 — Pull request merged — PEOPLE
- 10:04 — Deployed to all three environments · Done — SHIPPED

> “Normally a UI redesign takes months of designing, deciding and redesigning with several people. This was one developer doing the heavy lifting. It was only possible because of AI.”
>
> A team developer

Table layout → AI mockup → Shipped, matches

BUILT ON STORY C · 14–23 SEP

The same developer first moved the Dashboard onto the organization's design system and wrote a design-system skill that taught the agent how to use it.

The agent produced the artifacts; people made the decisions

AI AGENT PRODUCED	
`mockups/`	8 clickable pages, 2 iterations
`story.md`	22 acceptance criteria
`spec.md`	510 lines; 3 open questions, each with a recommendation
`plan.md`	2,190 lines; 14 tasks; 13 gaps and deviations resolved
`code`	43 files: +2,809 / −1,376 lines
`test-report.md`	140 unit and 29 UI tests passing; screenshots beside mockups
`review.md`	Adversarial self-review: no blocking findings, 2 issues fixed

PEOPLE DECIDED
Which mockup direction to build? **Iteration 2**
Drop a column from the list? **Yes**
Spinner delay: 3 seconds or 1? **Keep 3 seconds**
Add screenshot baselines now? **No new story**
One story or several? **One large story**
Accept the plan, and one known limit? **Approved**
Merge the pull request? **Merged**

Ten weeks in: 42 stories done, 99 pull requests, 32 releases

42
Stories done
of 97 written since 9 Aug

99
Pull requests merged
70 code changes · 29 story and spec updates

32
Releases
each deployed to all three environments

610
Commits
across four repositories, excluding merges

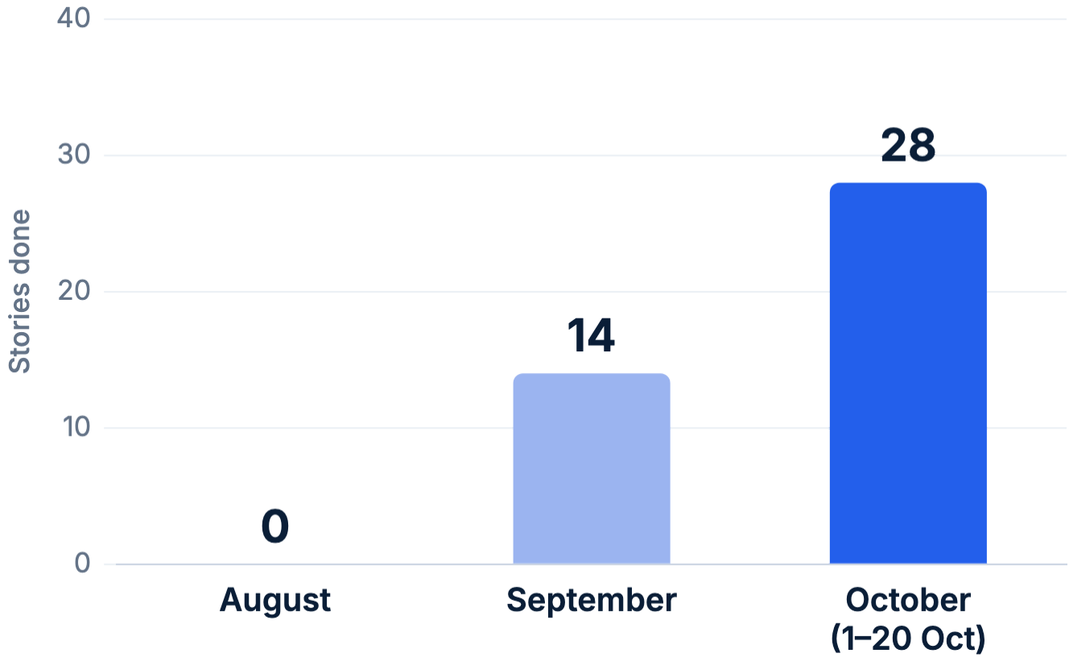
1.9 days
Median build time in October
down from 7.0 days in August

5
People
using two AI agents: Claude Code and Codex

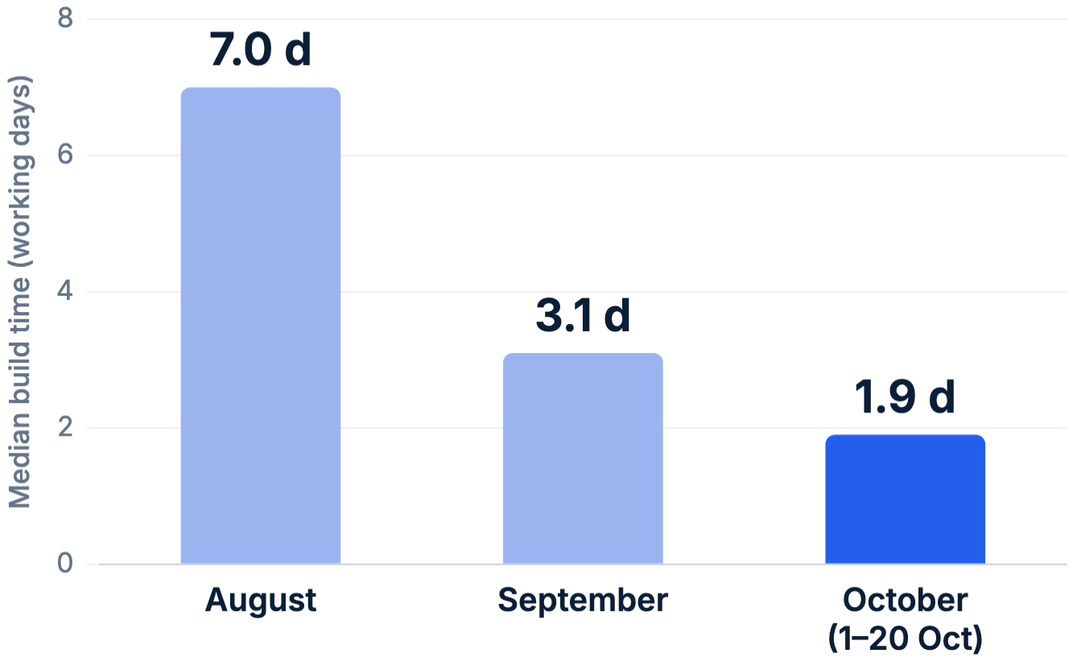
Git history and deploy tags, 9 Aug – 20 Oct 2026

Finished stories went from 0 to 14 to 28 a month, while build time fell from 7 days to under 2

STORIES FINISHED PER MONTH

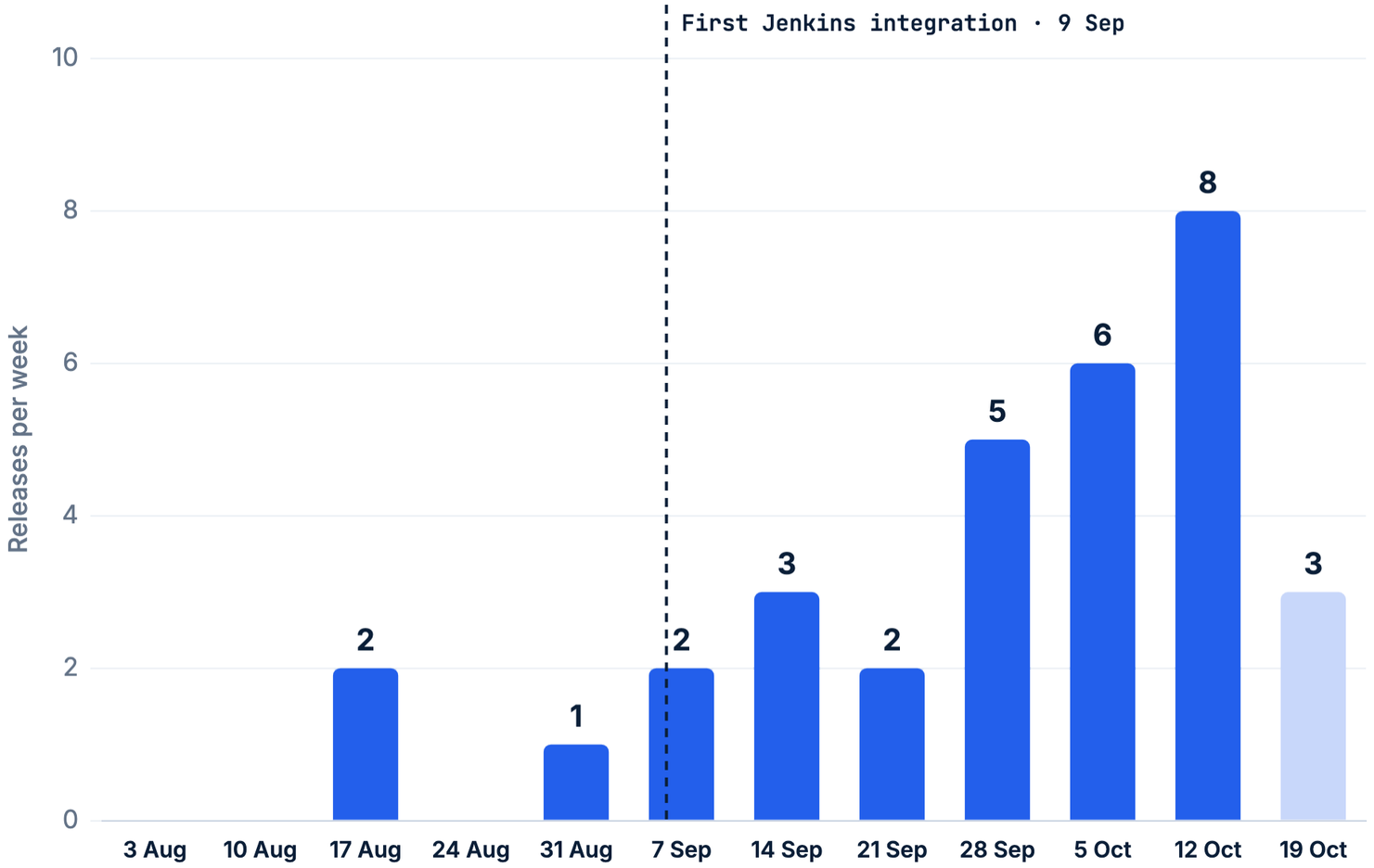


MEDIAN BUILD TIME (WORKING DAYS)



CAVEAT Build time runs from the first sign of work to the last implementation PR merged, and starts after the spec is approved. It does not include refinement. October's count covers 1–20 Oct.

Releases rose from 3 to 20 a month



3 → 9 → 20

releases in August, September and October (to the 20th)

96

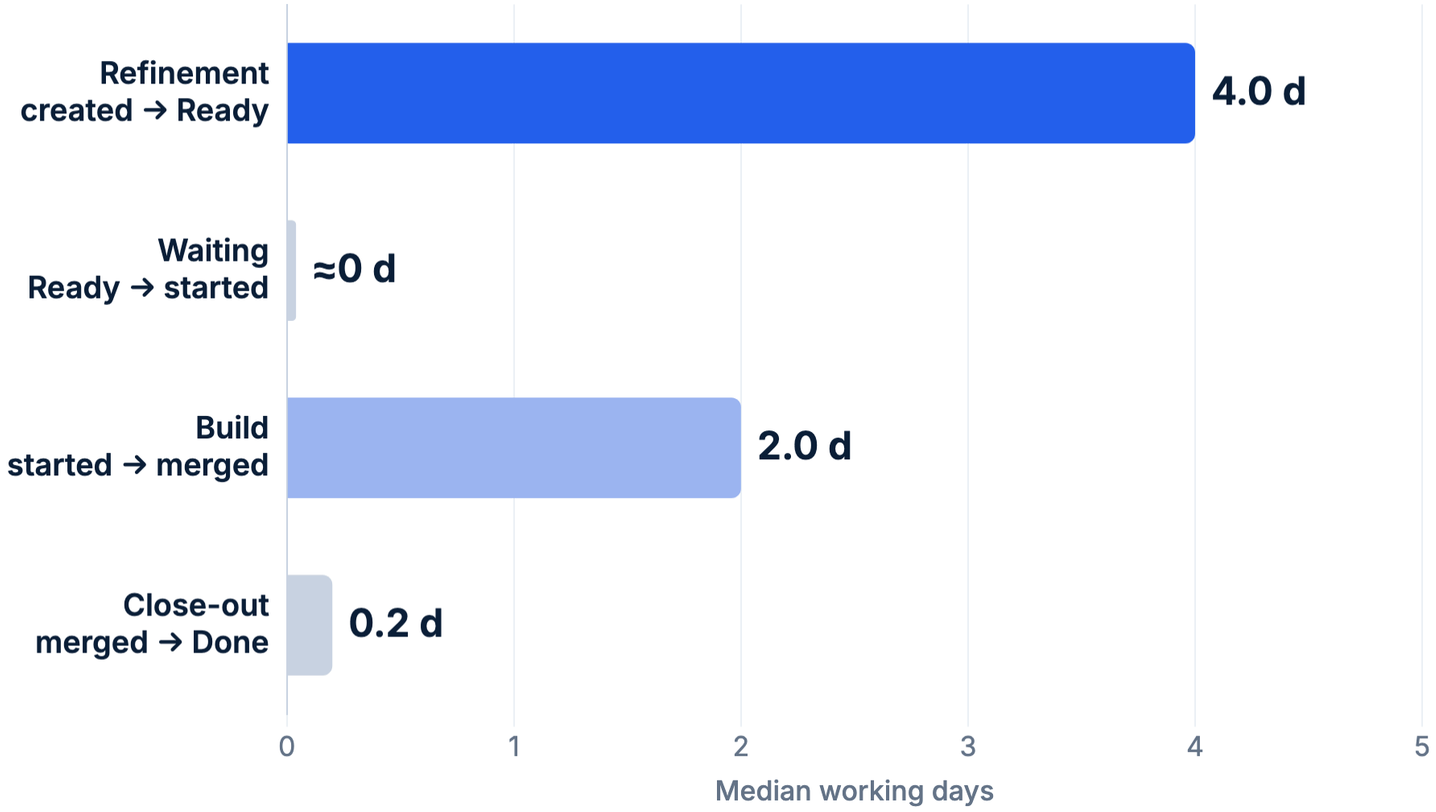
deployments: every release went to all three environments together

9 Sep

first successful Jenkins integration: the platform runs real automations

LESSON Get something simple through the client's CI early. Registrations and code analysis needed a person; once that was done, agents could turn out pull request after pull request.

Once a story is Ready, it's built in about two days; refinement takes twice as long



8.2 d

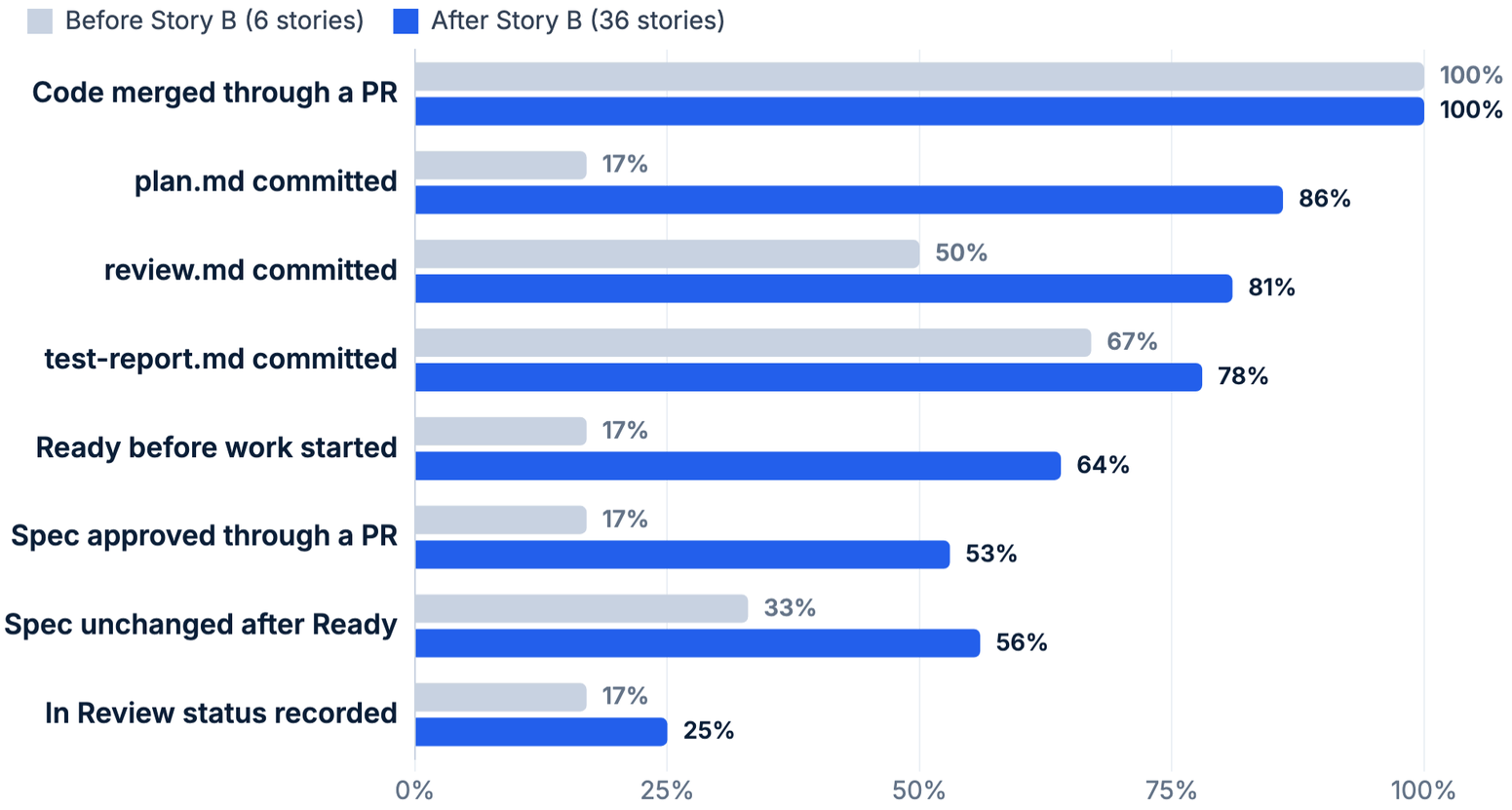
median lead time, from story created to Done

≈0

wait between Ready and the start of work. Anything Ready is picked up at once.

Medians for each stage don't add up to the lead time.

On paper vs in practice: the artifacts stuck, the status updates didn't



100%

of code merged through a pull request

25%

recorded In Review. Most jump from In Progress to Done in one commit.

56%

of specs stayed unchanged after Ready

Done stories: before Story B (n=6) vs after (n=36)

Code got cheap; clarity got expensive



38

stories waiting in refinement with no work started

12

stories in flight

2

Ready and waiting to be picked up

We built the process the way we built the product: in small, reviewed steps

August

9 Aug	Workspace and first agent guides
11 Aug	Simulated JIRA, story and spec templates
24 Aug	General-purpose agent skills adopted
25 Aug	First `create-jira` skill

September

3 Sep	**Story B:** the loop and lifecycle skills
9 Sep	**First successful Jenkins integration**
14 Sep	First domain skill: design system
21 Sep	Runner repository joins the process
29 Sep	User docs move into the platform

October

5 Oct	One skill source for Claude Code and Codex
12 Oct	Won't Do status
15 Oct	Documentation audience and depth rules

Process change Product milestone

Process changes ship as stories, with specs and reviews, like code

Story B · 3 Sep

The loop and lifecycle skills

Refinement → Ready → In Progress → In Review → Done, the three lifecycle skills, and spec approval through PRs.

Story C · 14 Sep

First domain skill

`design-system-ui` teaches agents the organization's design system.

Story E · 29 Sep

Docs move into the platform

User docs live in the product, with an LLM context page. The old wiki stand-in was retired.

Story D · 5 Oct

One skill source, two agents

`.claude/skills` is mirrored to Codex by a sync script.

Story F · 12 Oct

Won't Do

A terminal off-ramp with a date and a reason, so dropped work stays visible.

Story G · H · 15 Oct

Documentation depth

Rules for agents in `AGENTS.md`, plus a version for people in the in-app docs.

Process changes came from across the team, through the same loop as product work.

Trust grew gradually, and so did the risk of reviewing less

THE HONEST ARC

We started uncomfortable. The more the output looked "generally not wrong", the more comfortable we got, and the less closely each of us looked. Comfort is the part to watch.

WHAT KEPT IT SAFE

- Heavy involvement in structure and design up front
- Two human gates: the spec and the merge
- Adversarial self-review by the agent on every story
- Code analysis and security review before production
- An internal tool, used mostly in non-production

> “You can hand off your workload, but don't hand off your problem-solving.”
>
> From the team

Open question: how do you build trust in a tool like a self-driving car, handing over the wheel gradually, without ending up asleep at the wheel?

Four lessons about the process

01 Write the onboarding guide first

Agents follow `AGENTS.md` about as well as it's written. Short, specific and versioned beats long and vague.

02 Make the spec the contract

Put decisions in the spec before code. Then hold the plan, the code and the review to it.

03 Ask for evidence, not assertions

Test reports, screenshots and adversarial reviews make agent work quick to check.

04 Treat the process as code

Version it, review it, and change it through stories, so every change reaches every agent at once.

Three lessons about the organisation

05 Plan for the bottleneck to move

When code is cheap, refinement and review set the pace. Make time for decisions.

06 Get a simple app through the client's CI early

Registrations, code analysis and security review need a person. Once that path was open, agents could turn out pull request after pull request.

07 Hand off the workload, not the problem-solving

Agents draft, build and check. People still own the problem, the spec and the merge, and the specs they write.

Next: AI earlier in the process, and after failures

EARLIER IN THE PROCESS

AI for business analysts

Give BAs the same agent capability and context, so specs are drafted with AI from the start. The spec is where the time goes.

DECIDING WHAT TO TEST

Test planning, not just test running

Part of a spec is "what should we actually test?". We want AI to help build a coherent test plan, so we don't create a haystack of automations.

AFTER A FAILURE

Self-healing automations

When an automation fails, use AI to diagnose it and, later, to fix it.

Same rules apply: every AI feature goes through the loop, with an approved spec, a reviewed plan and evidence.

Questions for the room

01 Is this generic?

Would the loop and the two gates work on your project, or did we build something that only fits us?

02 Where would it break for you?

Which step would your team skip first, and what would you change?

03 What would you not trust an agent to do?

And how do you notice when you've started trusting it too much?

04 Who owns refinement on your team?

If code is cheap and clarity is expensive, who makes the decisions in time?

Discussion

Takeaway: agents write the code, people own the decisions, and when code is cheap, clarity is the constraint. The appendix has definitions, data notes and the skill list.

X BY 2®

How each number is measured

TERM	DEFINITION
Created	First commit that touches the story folder.
Ready	First commit whose story status is Ready (older stories: Approved).
Started	Earliest of: status In Progress, `plan.md` committed, or the first implementation commit naming the story.
Merged	The last implementation pull request for the story (spec PRs excluded).
Done	First commit with status Done. "Stories done" counts stories whose current status is Done.
Build time	Started → Merged.
Lead time	Created → Done.
Refinement	Created → Ready. Measured on every story that reached Ready.
Release	One build of `platform-app`, identified by its deploy tags for the three environments.
Lines of code	Source and test lines added on main, after the 9 Aug starting point. Merges, lockfiles, generated files, duplicates and moved code excluded.

Where the numbers come from, and what they can't tell you

SOURCES

- Git history of all four repositories: every branch and tag, 9 Aug – 20 Oct 2026
- 97 story folders in `jira/`, with status history read from each commit
- 96 deploy tags (32 releases × 3 environments)
- Merged pull requests read from the Git host's merge commits
- Team-level numbers only; this deck shows nothing per person

LIMITS

- PR approvals and reviewers live in the Git host, not in git
- Local work before the first commit is invisible, so build times are lower bounds
- Statuses are only as accurate as what people committed
- Commits carry no AI co-author marker, so AI involvement can't be measured per commit
- Story counts ignore size; lines of code measure output, not value

The ten workspace skills

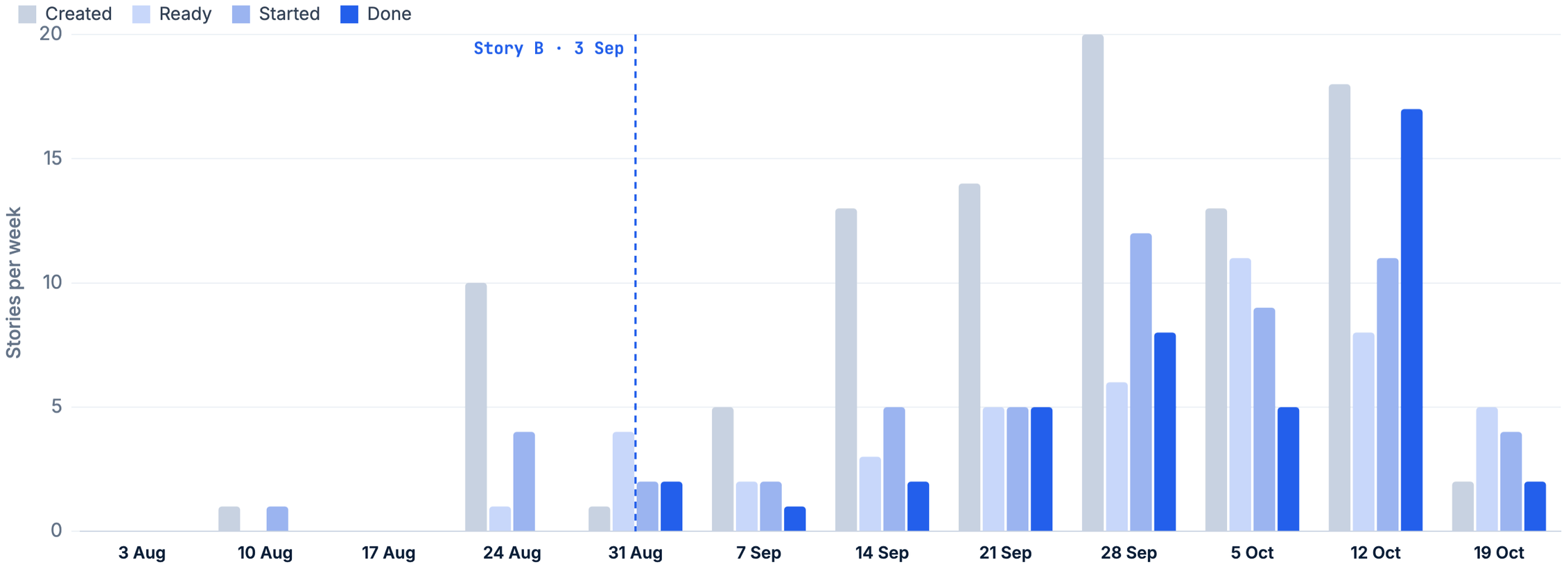
SKILL	TYPE	WHAT IT DOES
create-jira	lifecycle	Scopes and drafts a new story: folder, story.md and spec.md on a refinement branch, dependencies, team review.
next-jira	lifecycle	Finds the next unblocked Ready story for a repo, confirms the pick, writes plan.md for approval, then implements.
finish-jira	lifecycle	Runs verification, records the test report and review, opens the PR, updates docs, closes the story after merge.
writing-a-ticket	standard	User story, acceptance criteria, sizing and dependency links.
writing-a-spec	standard	Turns an idea into a spec the team can review, before any planning or code.
using-a-ticket-system	standard	Statuses, blocking, picking up the next ticket, and planning before doing.
writing-a-pr	standard	Branch naming, verification evidence, adversarial self-review and the PR description.
organizing-backend-ddd	standard	Domain-driven design for backend code: language, boundaries, layering and ADRs.
design-system-ui	domain	Organization design-system standards for building UI.
sdlc-bootstrap	setup	Installs the standards in a new project and interviews the team to generate its lifecycle skills.

Every process change, in order

DATE	STORY	CHANGE
9 Aug	-	Workspace repository created
11 Aug	-	Initial SDLC organization
12 Aug	Story I	Ticket IDs adopted
24 Aug	Story I	General-purpose agent skills adopted
25 Aug	Story I	First create-jira skill
3 Sep	Story B	SDLC restructured around wrapper and verb skills
9 Sep	Story D	Codex configuration added
14 Sep	Story C	First domain skill: design system

DATE	STORY	CHANGE
21 Sep	Story J	Runner repository brought into the process
29 Sep	Story E	Docs move into the platform; old wiki stand-in retired
3 Oct	Story K	Guidance corrected for three repositories
5 Oct	Story D	Claude-to-Codex skill sync script
12 Oct	Story F	Won't Do status added
15 Oct	Story G	Documentation audience and depth rules
15 Oct	Story H	Human version of the depth rules
19 Oct	Story L	Legacy repo-local Claude assets removed (in progress)

Stories created, made Ready, started and finished each week



Story milestones in git · weeks start Monday